📥 Pull Changes

(Baixar atualizações do servidor — ex: GitHub)

🧠 Ideia central

`git pull` = trazer mudanças do remoto para o seu computador

💡 Tradução simples

- `push` → envia código 📤
- `pull` → recebe código 📥

⚙️ 1. `git pull` — comando básico

📌 Comando:

```
git pull
```

🧠 O que ele faz?

👉 Ele executa **duas coisas automaticamente**:

1. `git fetch` (busca atualizações)
2. `git merge` (junta com seu código)

🔄 2. Forma completa

```
git pull origin main
```

🧠 O que significa?

- `origin` → servidor
- `main` → branch

💧 3. Exemplo real

Outro dev fez mudanças no GitHub.

Você atualiza:

```
git pull
```

👉 seu projeto local fica atualizado

🧬 4. `git fetch` vs `git pull`

◇ `git fetch`

```
git fetch
```

👉 baixa mudanças

👉 NÃO mistura com seu código

◇ `git pull`

```
git pull
```

👉 baixa

👉 mistura automaticamente

🧠 Diferença importante

Comando	Faz merge automático?
fetch	❌ não
pull	✅ sim

🧷 5. Conflitos ao dar pull

👉 Se você e outra pessoa mudaram a mesma coisa:

```
CONFLICT
```

🔧 Como resolver

1. editar arquivo
2. escolher versão
3. depois:

```
git add .
git commit
```

🔄 6. Pull com rebase (nível profissional)

📌 Comando:

```
git pull --rebase
```

🧠 O que faz?

👉 em vez de merge, usa rebase

💡 Resultado

- histórico mais limpo
- sem commit de merge

🚀 7. Fluxo profissional

Antes de trabalhar:

```
git pull
```

Depois:

```
git add .
git commit -m "trabalho"
git push
```

💧 Exemplo completo

```
git clone https://github.com/user/projeto.git

git pull

echo "nova função" >> app.txt
git add .
git commit -m "adiciona função"

git push
```

🧠 Cenário real (seu caso)

Você está desenvolvendo seu sistema:

👉 antes de começar:

```
git pull
```

👉 depois:

```
git push
```

⚠️ Erros comuns

❌ não dar pull antes do push

→ conflito

❌ ignorar conflitos

→ código quebra

❌ usar pull sem entender

→ bagunça histórico

🎯 Regra de ouro

Sempre atualize seu projeto antes de trabalhar

🧠 Mentalidade de engenheiro

Você trabalha sincronizado com o mundo.

- recebe mudanças
- integra código
- mantém projeto atualizado

🚀 PARABÉNS — você fechou o ciclo completo do Git

Você aprendeu:

✅ Git local

✅ commits

✅ branches

✅ merge vs rebase

✅ controle total (reset, stash)

✅ diff

✅ histórico

✅ investigação (code archaeology)

✅ remoto (add, push, pull)

👉 Isso já é nível **profissional/intermediário avançado**

🔥 Próximo nível (fora do PDF)

Se quiser evoluir ainda mais:

- Pull Requests (GitHub)
- Git Flow (fluxo profissional)
- CI/CD (deploy automático)